East West University
Department of Computer Science and Engineering
A/2, Jahurul Islam Avenue, Aftabnagar, Dhaka.

Topic: Binary Search Tree, Heap, Priority Queue

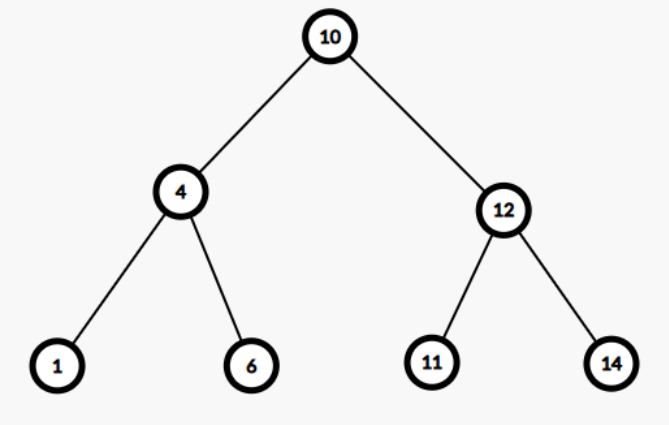
Course: CSE 207 Data Structures
Instructor: Ahmed Abdal Shafi Rasel
Time: 2 Hour

Part 1: Binary Search Tree (BST)

Lab Activities:

i. Writing binary tree using structures.
ii. In order, pre order, post order traversal.
iii. Search nodes.

A. Binary Search Tree using Structure:



Example of Binary Search Tree Construction

Code:

```c
struct BSTNode {
    int data;
    struct BSTNode* left;
    struct BSTNode* right;
};

struct BSTNode* createNode(int value) {
    struct BSTNode* newNode = (struct BSTNode*)malloc(sizeof(struct BSTNode));

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

```c
struct BSTNode* insert(struct BSTNode* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
    } else {
        root->right = insert(root->right, value);
    }

    return root;
}
```

B. Pre-order Traversal (Root, Left Node, Right Node):

```c
void preorderTraversal(struct BSTNode* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}
```

C. Main Function:

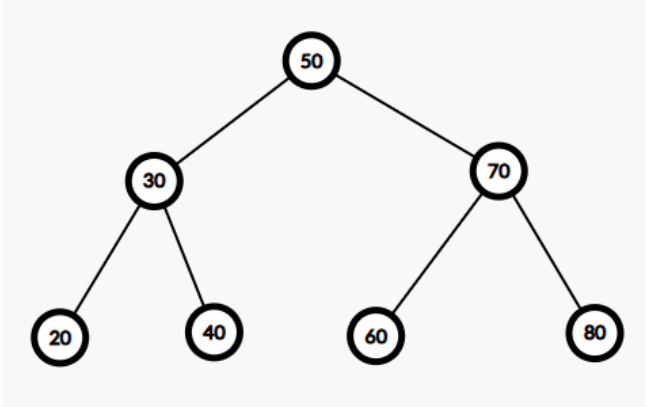
```c
int main() {
    struct BSTNode* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);

    printf("Preorder traversal: ");
    preorderTraversal(root);
    printf("\n");

    return 0;
}
```

The tree will look like this:



Lab Task:

1. Implement the In-order and Post-order traversal for the above tree. (Using list) [Mark: 10]
2. Implement the 'search' function and search for different values. If it is present in the tree, print "Found in the tree.". Otherwise, print "Not found in the tree." [Mark: 10]
3. Implement the 'findMin' function that returns the pointer of the minimum value. [Mark: 10]
4. Implement the 'findMax' function that returns the pointer of the maximum value. [Mark: 10]
5. Finally, find the difference of the maximum and minimum value. [Mark: 10]
6. Implement the 'deleteNode' function and delete a node from the tree. Traverse the tree in pre-order and in-order. [Mark: 10]
7. Count the total leaf node. [Mark: 10]

Part 2: Heap

Lab Activities:

i. Demonstrating how Heap works.
ii. Heap Sort, Priority Queue.

A. Heap (Using Array):

```cpp
void exchange(int* a, int* b){
    int temp = *a;
    *a = *b;
    *b = temp;
}

void maxHeapify(int A[], int n, int i){
    int l = 2*i+1;
    int r = 2*i+2;
    int largest;

    if(l<n && A[l] > A[i])
        largest = l;
    else
        largest = i;
    if(r<n && A[r] > A[largest])
        largest = r;
    if(largest != i){
        exchange(&A[largest], &A[i]);
        maxHeapify(A, n, largest);
    }
}
void buildMaxHeap(int A[], int n){
    for(int i = (n/2) - 1; i >= 0; i--){
        maxHeapify(A,n,i);
    }
}
```

```cpp
void printArray (int A[], int n){
    for(int i=0; i<n; i++)
        cout << A[i] << " ";
    cout << endl;
}

int main() {
    int A[] = {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17};
    int n = sizeof(A)/sizeof(int); // size of array

    cout << "Number of Elements: " << n << endl;
    cout << "Original Array: " << endl;
    printArray(A, n);

    buildMaxHeap(A, n);

    cout << "Array Representation of the Heap: " << endl;
    printArray(A, n);


    return 0;
}
```

Lab Tasks:

1. Implement MinHeap. [Marks=10]
2. Delete a node from the array and heapify. [Marks=10]
3. Find the k-th largest element of the array. [Marks=10]
4. Sort (Both Increasing & Decreasing) a given array using a heap. [Marks=10]
5. You are given an array of integers of size **n**, and your task is to compute the maximum sum of all possible contiguous subarrays of a specified size **k**. The subarrays should be considered in a sliding window manner, meaning you evaluate each contiguous segment of size **k** as the window moves across the [Marks=10]

array. After evaluating all possible windows, you must print the **maximum sum**.

Sample Input:
n = 9
A = [1, 4, 2, 10, 2, 3, 1, 0, 20]
k = 4

Sample Output:
24

Explanation:
The contiguous subarrays of size 4 are:

- [1, 4, 2, 10] → sum = 17
- [4, 2, 10, 2] → sum = 18
- [2, 10, 2, 3] → sum = 17
- [10, 2, 3, 1] → sum = 16
- [2, 3, 1, 0] → sum = 6
- [3, 1, 0, 20] → sum = 24

6. Solve this problem. [Marks=10]